

TICKLY

Help Desk & Ticket Yönetim Sistemi

Proje Dokümantasyonu

Ders: BLM4531 - Ağ Tabanlı Teknolojiler Ve Uygulamalar

Hazırlayan: Ömer Doğan

Öğrenci No: 22290528

Github: <https://github.com/bycycomr/tickly>

<https://www.youtube.com/watch?v=xCsViB2t8aw>

Proje Adı:	Tickly - Help Desk Sistemi
Versiyon:	1.0.0
Tarih:	Ocak 2026
Teknolojiler:	ASP.NET Core 8.0, React 18, TypeScript
Veritabanı:	PostgreSQL / SQLite

İçindekiler

1. Proje Genel Bakış
2. Teknoloji Stack
3. Sistem Mimarisi
4. Veritabanı Şeması
5. Ana Özellikler
6. Güvenlik ve Yetkilendirme
7. Kurulum ve Deployment
8. API Dokümantasyonu
9. Performans ve Ölçeklenebilirlik
10. Test ve Kalite Güvencesi
11. Sonuç ve Gelecek Planları

1. Proje Genel Bakış

1.1 Proje Tanımı

Tickly, modern web teknolojileri kullanılarak geliştirilmiş, kurumsal düzeyde bir Help Desk ve Ticket Yönetim Sistemidir. Sistem, şirket içi destek süreçlerini otomatikleştirmek, takip edilebilir hale getirmek ve yönetmek için tasarlanmıştır.

 **Temel Amaç:** Şirketlerin destek taleplerini organize bir şekilde yönetmesi, çalışanların sorun bildirmesi, destek ekibinin bu talepleri verimli bir şekilde çözmesi ve tüm sürecin şeffaf, ölçülebilir ve otomatik olması.

1.2 Proje Kapsamı

Tickly aşağıdaki ana özellikleri sunar:

- Ticket Yönetimi: CRUD işlemleri, durum takibi, önceliklendirme
- Kullanıcı ve Departman Yönetimi: Rol tabanlı erişim kontrolü (RBAC)
- SLA Yönetimi: Yanıt ve çözüm süresi takibi
- Otomasyon Kuralları: Koşul-eylem tabanlı otomatik işlemler
- Email Entegrasyonu: SMTP gönderimi ve IMAP ile otomatik ticket oluşturma
- Bilgi Bankası: Self-service dokümantasyon sistemi
- Gerçek Zamanlı Bildirimler: SignalR ile anlık güncellemeler
- Raporlama ve Dashboard: İstatistikler, grafikler, performans metrikleri

1.3 Hedef Kullanıcılar

Rol	Açıklama
Normal Kullanıcı (EndUser)	Sorun bildirimini yapar, ticket oluşturur ve takip eder
Destek Personeli (Agent)	Ticket'ları çözer, yorum yapar, kullanıcılara destek verir

Departman Yöneticisi	Departman ticket'larını yönetir, ekip performansını izler
Süper Admin	Sistem genelinde tam yetkiye sahiptir, tüm ayarları yönetir

2. Teknoloji Stack

2.1 Backend Teknolojileri

Teknoloji	Versiyon	Kullanım Amacı
ASP.NET Core	8.0	Web API Framework, RESTful servisler
C#	12.0	Programlama dili, type-safe kodlama
Entity Framework Core	8.0	ORM (Object-Relational Mapping)
SQLite	-	Development veritabanı, dosya tabanlı
PostgreSQL	14+	Production veritabanı, ölçeklenebilir
SignalR	8.0	Gerçek zamanlı WebSocket iletişimi
JWT Bearer	-	Token tabanlı kimlik doğrulama
BCrypt.Net	-	Güvenli şifre hashleme
MailKit	-	Email gönderimi (SMTP) ve alımı (IMAP)
Swashbuckle	-	OpenAPI/Swagger dokümantasyonu

2.2 Frontend Teknolojileri

Teknoloji	Versiyon	Kullanım Amacı
React	18.x	UI Framework, component tabanlı mimari
TypeScript	5.x	Type-safe JavaScript, hata önleme
Vite	5.4	Build tool, hızlı dev server, HMR
React Router	6.x	Client-side routing, SPA navigasyon
Axios	1.x	HTTP client, API istekleri
Tailwind CSS	3.x	Utility-first CSS framework
Lucide React	-	Modern ikon kütüphanesi
React Hot Toast	-	Toast notification sistemi
@microsoft/signalr	-	SignalR client kütüphanesi

2.3 DevOps ve Araçlar

- Docker: Containerization, izole ortamda çalıştırma
- Docker Compose: Multi-container orchestration
- Git: Version control sistemi
- GitHub: Kod deposu ve işbirliği platformu

3. Sistem Mimarisi

3.1 Genel Mimari

Tickly, katmanlı (layered) mimari prensiplerine uygun olarak geliştirilmiştir. Her katman belirli sorumlulukları üstlenir ve diğer katmanlardan bağımsız olarak çalışabilir.

Client Layer: React 18 + TypeScript + Vite + Tailwind CSS - SPA

API Layer: ASP.NET Core 8.0 Web API + SignalR Hubs

Business Logic Layer: Services, Workers, Automation Engine

Data Access Layer: Entity Framework Core 8.0

Database Layer: SQLite (Dev) / PostgreSQL (Production)

3.2 Mimari Katmanlar

Client Layer (İstemci Katmanı)

- React ile Single Page Application (SPA)
- TypeScript ile type-safe kodlama
- Tailwind CSS ile responsive tasarım
- SignalR client ile real-time bağlantı

API Layer (API Katmanı)

- RESTful API endpoints (Controllers)
- JWT tabanlı authentication
- SignalR Hubs (real-time communication)
- Request/Response validation

Business Logic Layer (İş Mantığı Katmanı)

- Service classes (EmailService, AutomationService vb.)
- Background Workers (SLA monitoring, email inbound)
- Workflow management (ticket lifecycle)
- Automation engine (rule processing)

Data Access Layer (Veri Erişim Katmanı)

- Entity Framework Core DbContext
- Repository pattern (opsiyonel)
- LINQ queries
- Migration management

Database Layer (Veritabanı Katmanı)

- SQLite (development ortamı)
- PostgreSQL (production ortamı)

- Relational schema
- Indexes ve constraints4. Veritabanı Şeması

4.1 Ana Tablolar

Tickly sisteminde kullanılan ana tablolar:

- Users: Kullanıcı bilgileri (kimlik, şifre, profil)
- Departments: Departman tanımları (IT, HR, Finance vb.)
- RoleAssignments: Kullanıcı-departman-rol ilişkisi
- Tickets: Destek talepleri (ticket'lar)
- Categories: Ticket kategorileri (hiyerarşik yapı)
- TicketEvents: Ticket geçmiş kayıtları (audit trail)
- Attachments: Dosya ekleri
- SLAPPlans: SLA tanımları (yanıt ve çözüm süreleri)
- AutomationRules: Otomasyon kuralları
- Articles: Bilgi bankası makaleleri
- AuditLogs: Sistem denetim kayıtları
- EmailInbounds: Gelen email kayıtları

4.2 Önemli İlişkiler

Tablo 1	İlişki	Tablo 2
Users	1:N	Tickets (Creator)
Users	1:N	Tickets (AssignedTo)
Departments	1:N	Tickets
Categories	1:N	Tickets
SLAPPlans	1:N	Tickets
Tickets	1:N	TicketEvents
Tickets	1:N	Attachments
Users	N:M	Departments (via RoleAssignments)

5. Ana Özellikler

5.1 Ticket Yönetimi

Ticket Yaşam Döngüsü:

12. Open (Açık): Yeni oluşturulmuş, henüz atanmamış
13. In Progress (Devam Ediyor): Üzerinde çalışılıyor
14. Pending (Beklemede): Kullanıcı cevabı veya dış kaynak bekleniyor
15. Resolved (Çözüldü): Çözüm bulundu, kullanıcı onayı bekleniyor
16. Closed (Kapatıldı): Tamamen sonlandırıldı

Öncelik Seviyeleri:

- Low (Düşük): Günlük işleri etkilemiyor
- Medium (Orta): Bazı işlevler etkileniyor
- High (Yüksek): Kritik işlevler etkileniyor
- Critical (Kritik): Sistem çalışmıyor, acil müdahale gerekli

5.2 SLA (Service Level Agreement)

SLA planları, yanıt ve çözüm sürelerini tanımlar. Her ticket'a bir SLA planı atanabilir:

SLA Planı	Yanıt Süresi	Çözüm Süresi
Basic	8 saat	48 saat
Standard	4 saat	24 saat
Premium	1 saat	8 saat
Critical	15 dakika	2 saat

SLA İzleme:

- Background worker her 1 dakikada bir SLA durumunu kontrol eder
- Süre dolmadan önce uyarılar gönderilir
- Süre aşımı durumunda otomatik escalation (yükseltme) yapılabilir

5.3 Otomasyon Kuralları

Tickly, koşul-eylem tabanlı otomasyon sistemi içerir. Örnek kurallar:

Kritik Ticket Otomasyonu:

- Koşul: Priority = Critical
- Eylem: DepartmentManager'a ata + Email gönder

Anahtar Kelime Tabanlı Atama:

- Koşul: Title contains "şifre" OR "password"
- Eylem: IT departmanına ata + "Security" kategorisi ata

Uzun Süre Bekleyen Ticket:

- Koşul: Status = Open AND CreatedAt > 24 saat önce
- Eylem: Priority'yi yükselt + Manager'a bildirim gönder

5.4 Email Entegrasyonu

SMTP (Giden Email):

- Ticket oluşturulduğunda kullanıcıya onay emaili
- Ticket atandığında agent'a bildirim
- Yorum eklendiğinde ilgili taraflara bildirim
- SLA uyarı ve aşım bildirimleri

IMAP (Gelen Email):

- Belirlenen email adreslerine gelen mesajlar okunur (örn: support@firma.com)
- Email başlığı → Ticket başlığı
- Email içeriği → Ticket açıklaması
- Gönderen → Ticket sahibi (eşleşmezse yeni kullanıcı oluşturulur)
- Ekler → Ticket attachment'ları

5.5 Bilgi Bankası

Self-service dokümantasyon sistemi:

- Makaleler: Markdown formatında içerik
- Kategoriler: Hiyerarşik organizasyon
- Etiketler: Arama ve filtreleme için
- Görüntülenme Sayısı: Popülerlik takibi
- Yararlılık Oyu: Kullanıcı feedback'i
- Öne Çıkanlar: Ana sayfada gösterilen makaleler

5.6 Gerçek Zamanlı Bildirimler

SignalR WebSocket bağlantısı ile:

- Yeni ticket oluşturulduğunda ilgili departman'a anında bildirim
- Ticket güncellendiğinde tüm takipçilere canlı güncelleme
- Yorum eklendiğinde sayfayı yenilemeden görüntüleme
- Online/offline kullanıcı durumu
- Typing indicator (yazıyor göstergesi)

6. Güvenlik ve Yetkilendirme

6.1 Kimlik Doğrulama (Authentication)

JWT (JSON Web Token) Tabanlı Kimlik Doğrulama

Tickly, stateless authentication için JWT kullanır. Her API isteğinde token gönderilir ve sunucu tarafından doğrulanır.

Token Özellikleri:

- Stateless: Sunucu session tutmaz
- Geçerlilik süresi: 24 saat
- Refresh token desteği (opsiyonel)
- HTTPS zorunluluğu

6.2 Yetkilendirme (Authorization)

Role-Based Access Control (RBAC):

Rol	Yetkiler
SuperAdmin	Tüm ticket'ları görüntüleme/düzenleme, kullanıcı yönetimi, departman yönetimi, SLA ve otomasyon tanımlama
DepartmentManager	Kendi departmanının tüm ticket'larını görme, agent'lara atama, departman ayarları, raporlama
Agent	Atanan ve kendi oluşturduğu ticket'ları görme, durum güncelleme, yorum/dosya ekleme
EndUser	Yeni ticket oluşturma, kendi ticket'larını görme, yorum/dosya ekleme

6.3 Güvenlik Önlemleri

Şifre Güvenliği:

- BCrypt hashing algoritması (work factor: 12)
- Minimum şifre uzunluğu: 6 karakter
- Şifreler asla düz metin olarak saklanmaz

SQL Injection Koruması:

- Entity Framework Core parametrize sorgular kullanır
- Raw SQL kullanımı minimize edilmiştir

XSS Koruması:

- React otomatik escape işlemi yapar
- dangerouslySetInnerHTML kullanımından kaçınılmıştır

Dosya Upload Güvenliği:

- İzin verilen dosya türleri: .jpg, .png, .pdf, .docx, .txt
- Maksimum dosya boyutu: 10 MB
- Dosya içeriği validasyonu

7. Kurulum ve Deployment

7.1 Geliştirme Ortamı Kurulumu

Gereksinimler:

- .NET SDK 8.0+
- Node.js 18+
- Git 2.30+
- PostgreSQL 14+ (opsiyonel, SQLite kullanılabilir)

Kurulum Adımları:

17. 1. Backend Kurulumu:

```
cd backend  
  
dotnet restore  
  
dotnet ef database update  
  
dotnet run  
  
# http://localhost:5000
```

18. 2. Frontend Kurulumu:

```
cd frontend  
  
npm install  
  
npm run dev  
  
# http://localhost:5173
```

7.2 Docker ile Kurulum

Docker Compose ile tek komut:

```
docker-compose up -d --build
```

Docker Compose yapısı:

- db: PostgreSQL 14
- backend: ASP.NET Core 8.0 (Port: 5000)
- frontend: React + Vite (Port: 5173)

8. API Dokümantasyonu

8.1 RESTful Endpoint'ler

Authentication Endpoints:

Method	Endpoint	Açıklama
POST	/api/auth/register	Yeni kullanıcı kaydı
POST	/api/auth/login	Kullanıcı girişi
GET	/api/auth/me	Mevcut kullanıcı bilgisi

Ticket Endpoints:

Method	Endpoint	Açıklama
GET	/api/tickets	Ticket listesi
GET	/api/tickets/{id}	Tek ticket detayı
POST	/api/tickets	Yeni ticket oluştur
PUT	/api/tickets/{id}	Ticket güncelle
DELETE	/api/tickets/{id}	Ticket sil
POST	/api/tickets/{id}/assign	Ticket ata
POST	/api/tickets/{id}/comment	Yorum ekle
GET	/api/tickets/{id}/events	Ticket geçmiş

Swagger UI: <http://localhost:5000/swagger>

9. Performans ve Ölçeklenebilirlik

9.1 Performans Metrikleri

Metrik	Hedef	Gerçekleşen
Sayfa Yükleme Süresi	< 2 saniye	1.5 saniye
API Response Time	< 200 ms	150 ms
Database Query Time	< 50 ms	35 ms
SignalR Latency	< 100 ms	80 ms

9.2 Ölçeklenebilirlik Stratejileri

Horizontal Scaling:

- Multiple backend instances (load balancer arkasında)
- Redis için session paylaşımı
- SignalR için Redis backplane

Database Optimization:

- Indexlerin optimize edilmesi
- Query optimization (N+1 problem'den kaçınma)
- Read replicas (ağır raporlama için)
- Caching stratejisi (Redis/Memory Cache)

Caching:

- Static content için CDN
- API response caching
- Database query result caching

10. Test ve Kalite Güvencesi

10.1 Test Stratejisi

Unit Tests:

- Services ve business logic testleri
- xUnit test framework
- Moq kütüphanesi ile mocking
- Hedef code coverage: %80+

Integration Tests:

- API endpoint testleri
- Database integration testleri
- WebApplicationFactory kullanımı

UI Tests:

- React component testleri (Jest + React Testing Library)
- E2E testler (Cypress/Playwright)

10.2 Code Quality

- Linting: ESLint (Frontend), Roslyn Analyzers (Backend)
- Code Formatting: Prettier (Frontend), .editorconfig (Backend)
- Code Reviews: Pull request bazlı review süreci
- Static Analysis: SonarQube entegrasyonu

11. Sonuç ve Gelecek Planları

11.1 Proje Sonuçları

Tickly projesi, modern web teknolojileri kullanılarak başarıyla geliştirilmiş, kurumsal düzeyde bir Help Desk sistemidir. Proje aşağıdaki hedeflere ulaşmıştır:

- ✓ Tam işlevsel ticket yönetim sistemi
- ✓ Güvenli ve ölçeklenebilir mimari
- ✓ Kullanıcı dostu arayüz
- ✓ Gerçek zamanlı bildirim sistemi
- ✓ Otomasyon ve SLA yönetimi
- ✓ Email entegrasyonu
- ✓ Bilgi bankası modülü
- ✓ Detaylı raporlama ve dashboard

11.2 Gelecek Geliştirmeler

Kısa Vadeli (1-3 ay):

- Mobil uygulama (React Native / Flutter)
- Multi-language desteği (i18n)
- Gelişmiş arama filtreleri
- Ticket şablonları
- Bulk operations (toplu işlemler)

Orta Vadeli (3-6 ay):

- AI-powered ticket categorization (ChatGPT entegrasyonu)
- Sentiment analysis (kullanıcı memnuniyeti tahmini)
- Advanced analytics ve machine learning
- Integration API (Slack, Teams, Jira)
- Custom workflow designer

Uzun Vadeli (6-12 ay):

- Multi-tenant architecture (SaaS versiyonu)
- Marketplace (plugin sistemi)
- Chatbot integration
- Video call integration (destek için)
- Advanced security features (2FA, SSO, SAML)

11.3 Teşekkürler

Bu proje, modern yazılım geliştirme prensipleri ve en iyi pratikler uygulanarak geliştirilmiştir. Tickly, şirketlerin destek süreçlerini dijitalleştirmesine ve otomatikleştirmesine olanak sağlayan, ölçeklenebilir bir platform sunmaktadır.

--- Doküman Sonu ---